

AWS WAF & Shield — Web Application Firewall and DDoS Protection

Detailed Explanation

Overview: Web Application Security in AWS

AWS provides two complementary services for protecting web applications from security threats:

1. **AWS WAF (Web Application Firewall):** Application-layer (Layer 7) filtering service that monitors and controls HTTP/HTTPS web traffic
2. **AWS Shield:** Managed DDoS (Distributed Denial of Service) protection service at infrastructure layers (Layer 3, 4, and 7)

Both services work together to provide comprehensive protection against application-level attacks, DDoS attacks, and malicious traffic patterns.

AWS WAF (Web Application Firewall)

What is AWS WAF?

AWS WAF is a web application firewall that helps protect your web applications from common web-based attacks by allowing you to control which traffic reaches your applications. It monitors HTTP/HTTPS web requests and matches them against rules you define[1].

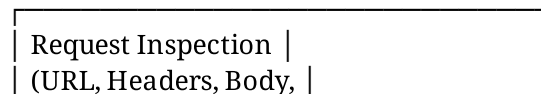
Key Characteristics:

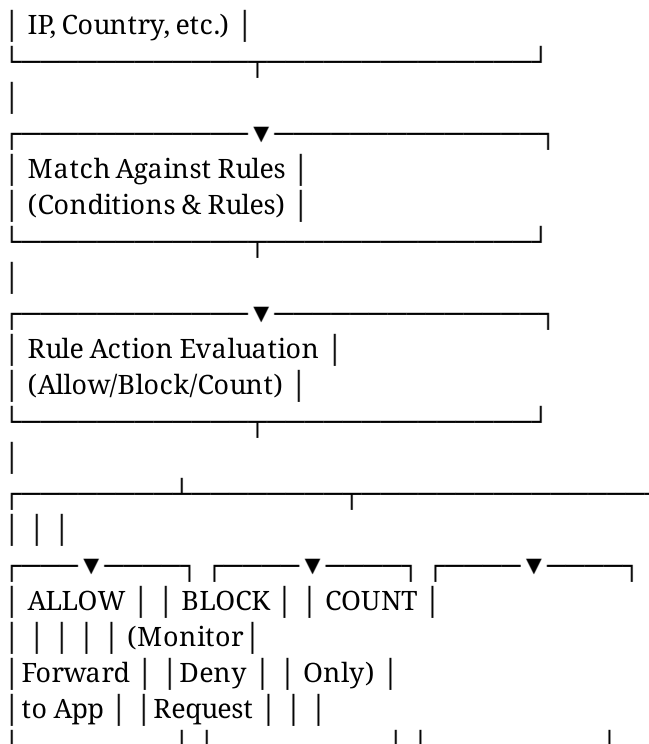
- **Layer 7 Protection:** Operates at the application layer
- **Flexible Rule Creation:** Define custom rules based on IP address, country, HTTP header, query string, URI path, and more
- **Managed Rule Groups:** Pre-built rules covering OWASP Top 10 and common vulnerabilities
- **Real-time Filtering:** Inspects requests before they reach your application
- **Integration:** Works with CloudFront, Application Load Balancer (ALB), API Gateway, AppSync
- **Granular Control:** Allow, block, or count requests matching specific patterns
- **Cost Model:** Pay per web ACL, per rule, and per million requests processed
- **Logging:** Send request logs to CloudWatch, S3, or Kinesis Data Firehose

AWS WAF Architecture:

Web Request (HTTP/HTTPS)

↓





Web ACL (Web Access Control List)

A Web ACL is a collection of rules that defines how to process web requests.

Components:

Web ACL

- └ Default Action (Allow or Block)
- └ Rules
 - └ Rule 1: Block if IP is in IP reputation list
 - └ Rule 2: Block if SQL injection pattern detected
 - └ Rule 3: Rate limit if more than 2000 requests/5 min
 - └ Rule N: Custom rule based on business logic
- └ Capacity Units: Each Web ACL and rule consumes capacity
- └ Associated Resources:
 - └ CloudFront distributions
 - └ Application Load Balancers
 - └ API Gateways
 - └ AppSync APIs

Example Web ACL Definition:

```

{
  "Name": "ProductionWebACL",
  "DefaultAction": {
    "Allow": {}
  },
  "Rules": [
    {
      "Name": "BlockSQLInjection",

```

```
"Priority": 1,
"Statement": {
  "ManagedRuleGroupStatement": {
    "VendorName": "AWS",
    "Name": "AWSManagedRulesSQLiRuleSet"
  }
},
"OverrideAction": {
  "None": {}
},
"VisibilityConfig": {
  "SampledRequestsEnabled": true,
  "CloudWatchMetricsEnabled": true,
  "MetricName": "SQLiRuleSetMetric"
},
{
  "Name": "BlockXSS",
  "Priority": 2,
  "Statement": {
    "ManagedRuleGroupStatement": {
      "VendorName": "AWS",
      "Name": "AWSManagedRulesCommonRuleSet",
      "RuleActionOverrides": [
        {
          "Name": "CrossSiteScripting_BODY",
          "ActionToUse": {
            "Block": {}
          }
        }
      ]
    }
  },
  "OverrideAction": {
    "None": {}
  },
  "VisibilityConfig": {
    "SampledRequestsEnabled": true,
    "CloudWatchMetricsEnabled": true,
    "MetricName": "XSSRuleSetMetric"
  },
  {
    "Name": "RateLimitRule",
    "Priority": 3,
    "Statement": {
      "RateBasedStatement": {
        "Limit": 2000,
        "AggregateKeyType": "IP"
      }
    }
  },
}
```

```

"Action": {
  "Block": {}
},
"VisibilityConfig": {
  "SampledRequestsEnabled": true,
  "CloudWatchMetricsEnabled": true,
  "MetricName": "RateLimitMetric"
}
},
{
  "Name": "AllowWhitelistedIPs",
  "Priority": 4,
  "Statement": {
    "IPSetReferenceStatement": {
      "ARN": "arn:aws:wafv2:us-east-1:123456789012:global/ipset/trusted-partners/a1234567-a123-a123-a123-a123456789ab"
    }
  },
  "Action": {
    "Allow": {}
  },
  "VisibilityConfig": {
    "SampledRequestsEnabled": true,
    "CloudWatchMetricsEnabled": true,
    "MetricName": "WhitelistMetric"
  }
}
],
"Capacity": 100,
"VisibilityConfig": {
  "SampledRequestsEnabled": true,
  "CloudWatchMetricsEnabled": true,
  "MetricName": "ProductionWebACLMetric"
}
}

```

WAF Rules and Conditions

Rule Types:

1. **Regular Rules** - Evaluate requests based on conditions
 - Condition: IP address, Country, Request header, Query string, URI path
 - Action: Allow, Block, or Count
2. **Rate-Based Rules** - Limit requests from IP addresses
 - Track requests per IP over 5-minute window
 - Minimum rate limit: 100 requests/5 minutes
 - Use case: Prevent brute force attacks, API abuse
3. **Managed Rules** - Pre-built rule groups maintained by AWS or marketplace vendors
 - AWS Managed Rules (most common, lower cost)
 - AWS Marketplace Rules (specialty, vendor-maintained)

Common Conditions:

Condition Type	Description	Example
IP Match	Allow/block based on IP address or CIDR block	Block 192.0.2.0/24
Country	Filter requests by geographic origin	Allow only US/CA/UK
Request Header	Inspect HTTP headers	Block if User-Agent = Bot
Query String	Inspect URL query parameters	Block if contains '; DROP TABLE
URI Path	Inspect request path	Block if path = /admin
Request Body	Inspect POST/PUT body	Detect SQL injection patterns
Byte Match	Match specific byte sequences	Detect malicious payloads
Regular Expression	Use regex patterns	Match complex attack signatures

Managed Rule Groups

AWS Managed Rules provide pre-built protection against common threats[2].

Available Rule Groups:

1. Core Rule Set (AWSManagedRulesCommonRuleSet)

- Covers OWASP Top 10 vulnerabilities
- XSS, CSRF, Local File Inclusion (LFI), Remote File Inclusion (RFI)
- PHP Injection, Cross-Site Tracing (XST)
- Rules: ~200+, Regular updates

2. Known Bad Inputs (AWSManagedRulesKnownBadInputsRuleSet)

- Blocks requests for known vulnerability patterns
- Log4j RCE, CVE exploits
- Exploitable paths, known malicious payloads

3. SQL Injection (AWSManagedRulesSQLiRuleSet)

- Detects SQL injection patterns
- Checks query strings, body, cookies, headers

- Different SQL dialects (MySQL, PostgreSQL, MSSQL, Oracle)

4. Amazon IP Reputation List (AWSManagedRulesAmazonIpReputationList)

- Blocks requests from known malicious IPs
- Botnets, known attacker IPs
- Continuously updated by AWS threat intelligence

5. Bot Control (AWSManagedRulesBotControlRuleSet)

- Identifies and blocks malicious bots
- Categories: Advertising bots, scrapers, security tools
- Additional cost: ~\$10/month for 1M requests

6. Account Takeover Prevention (AWSManagedRulesATPRuleSet)

- Protects against credential compromise
- Detects compromised credentials, brute force, credential stuffing
- Additional cost: ~\$10/month for 1M requests

7. Fraud Control (AWSManagedRulesFCRuleSet)

- Detects suspicious activities like account takeover
- Token-based fraud detection
- Additional cost included in Fraud Control pricing

WAF vs WAF Classic

Feature	WAF (Current)	WAF Classic (Deprecated)
Capacity Units	Yes (modern approach)	No
Logging	✓ CloudWatch, S3, Kinesis	Limited
Managed Rules	✓ Extensive	Limited
Bot Control	✓ Yes	No
Automation	✓ Better	No
API Support	✓ Full	Limited
Performance	✓ Better	Legacy

AWS Shield

What is AWS Shield?

AWS Shield is a managed DDoS protection service that safeguards applications running on AWS. It provides automatic inline mitigation against DDoS attacks at network layers 3, 4, and application layer 7[3].

Key Characteristics:

- **Layer 3, 4, 7 Protection:** Network, Transport, Application layers
- **Two Tiers:** Standard (free) and Advanced (subscription)
- **Automatic Mitigation:** Inline protection without configuration
- **Global Deployment:** AWS edge locations provide attack absorption
- **Integration:** Works with CloudFront, Route 53, ELB, ALB, NLB, AppSync
- **Attack Notification:** Alerts and metrics for detected attacks
- **DDoS Cost Protection:** Shield Advanced includes DDoS-related data transfer costs
- **Response Team:** Shield Advanced includes DDoS Response Team (DRT)

AWS Shield Architecture:

User Requests

↓

AWS Global Network

↓

AWS Shield Protection Layer	
• Layer 3 (Network)	
- UDP Floods	
- DNS Query Floods	
- SYN Floods	
• Layer 4 (Transport)	
- Volumetric attacks	
- Protocol attacks	
- Large packet floods	
• Layer 7 (Application)	
- HTTP floods	
- Slowloris attacks	
- Cache-busting attacks	

↓

Protected Resources

(CloudFront, Route 53, ALB, etc.)

Shield Standard vs Shield Advanced

Shield Standard (Free for all AWS customers):

Feature	Availability
Network/Transport Layer Protection	✓
Application Layer Monitoring	✓
Automatic Attack Detection	✓
Real-time Metrics	✓
DDoS Response Team (DRT)	✗
Advanced WAF Integration	✗
Cost Protection	✗
Predictive Scaling	✗
Custom Mitigations	✗
Firewall Manager Integration	✗

Table 1: Shield Standard Features

Shield Advanced (\$3,000/month, 12-month commitment):

Feature	Availability
All Shield Standard Features	✓
Advanced Layer 7 Detection	✓
DDoS Response Team (DRT)	✓
WAF Protection Included	✓ (WAF fees waived)
DDoS Cost Protection	✓
Real-time Attack Diagnostics	✓
Protected Resource Limits	Up to 500 resources
Automatic AWS WAF Deployment	✓
Predictive Scaling	✓
Firewall Manager Integration	✓

Table 2: Shield Advanced Features

DDoS Attack Types and Mitigation

Layer 3/4 Attacks (Network/Transport):

Attack Type	Description	Shield Mitigation
UDP Floods	Overwhelming target with UDP packets	Connection rate limiting, traffic filtering
SYN Floods	Overwhelming target with SYN requests	TCP connection rate limiting, SYN cookies
DNS Query Floods	Massive DNS queries to amplify attack	DNS request rate limiting, query filtering
NTP Amplification	Abuse NTP servers to amplify attack	Source validation, filtering
ICMP Floods	Overwhelming target with ICMP packets	ICMP rate limiting, filtering

Layer 7 Attacks (Application):

Attack Type	Description	Shield/WAF Mitigation
HTTP Floods	High volume HTTP requests	Rate limiting, connection tracking, behavioral analysis
Slowloris	Slow HTTP requests to exhaust connections	Connection timeout tuning, request timeout
Cache-Busting	Requests designed to bypass cache	CloudFront caching rules, WAF rules
Application-Level Floods	Requests to specific endpoints	Rate-based rules, query string analysis
XML Bomb	Large XML payloads to exhaust resources	Request body size limits, WAF inspection

DDoS Response Team (DRT)

Available with Shield Advanced.

Services Provided:

- **24/7 Expert Support:** Access to AWS DDoS specialists
- **Attack Analysis:** Real-time analysis during active attacks
- **Custom WAF Rules:** DRT can create custom WAF rules during attack
- **Architectural Review:** Pre-attack security assessment

- **Response Planning:** Help develop DDoS response procedures
- **Crisis Communication:** Coordinate response during major attacks

Feature Comparison Matrix

Feature	AWS WAF	AWS Shield Standard	AWS Shield Advanced
Purpose	Application firewall	Layer 3/4 DDoS protection	Layer 3/4/7 DDoS + WAF
Layer	Layer 7 (Application)	Layers 3-4 (Network/Transport)	Layers 3-4-7 (All)
Cost	~\$5-20/month (depends on usage)	Free	\$3,000/month
Rules	Custom + Managed	N/A	Included (WAF)
Attack Detection	Pattern-based	Automatic	Automatic + Predictive
Response Team	No	No	Yes (24/7)
Cost Protection	No	No	Yes (DDoS-related costs)
Real-time Mitigation	Yes	Yes (automatic)	Yes (more aggressive)
Custom Waf Rules	Yes	N/A	Yes (DRT can add)
Scope	Application-specific	Global network	Global network + App
Best For	Brute force, SQL injection, XSS, bots	Volumetric attacks	Mission-critical apps
Integration	ALB, CloudFront, API Gateway, AppSync	CloudFront, Route 53, ELB	All Shield Standard + WAF

Pricing Model

AWS WAF Pricing:

Total Monthly Cost =
(Web ACLs × \$5) +
(Rules × \$1) +
(Requests in millions × \$0.60)

Example Calculation:

- 3 Web ACLs: $3 \times \$5 = \15
- 15 rules: $15 \times \$1 = \15
- 50 million requests: $50 \times \$0.60 = \30
- **Total: \$60/month**

AWS Shield Advanced with WAF:

Total Monthly Cost =
\$3,000 (Shield Advanced) +
AWS WAF costs (Rules/Requests) +
Bot/Fraud Control (if enabled)

Key Points:

- Shield Advanced WAF fees are waived (\$0 WAF cost)
- Bot Control: ~\$10/month per 1M requests
- Fraud Control: Additional subscription fee
- Data transfer protection: Included up to 2TB

Detailed Examples

Example 1: Implementing AWS WAF for E-commerce Application

Scenario: E-commerce company needs to protect customer login page from brute force attacks, SQL injection, and malicious bots.

Step 1: Create IP Set for Threat Intelligence

Create IP set to block known malicious IPs

```
aws wafv2 create-ip-set
--name BlockedIPSet
--scope REGIONAL
--region us-east-1
--ip-address-version IPV4
--addresses ['192.0.2.0/24', '198.51.100.0/24']
```

```
IP_SET_ARN=$(aws wafv2 list-ip-sets
--scope REGIONAL
--region us-east-1
```

```
--query 'IPSets[0].ARN'
--output text)
```

Step 2: Create Web ACL with Multiple Rules

Create Web ACL with AWS managed rules

```
aws wafv2 create-web-acl
--name EcommerceWebACL
--scope REGIONAL
--default-action Allow={}
--rules '[
{
  "Name": "BlockKnownBadInputs",
  "Priority": 1,
  "Statement": {
    "ManagedRuleGroupStatement": {
      "VendorName": "AWS",
      "Name": "AWSManagedRulesKnownBadInputsRuleSet"
    }
  },
  "OverrideAction": {
    "None": {}
  },
  "VisibilityConfig": {
    "SampledRequestsEnabled": true,
    "CloudWatchMetricsEnabled": true,
    "MetricName": "KnownBadInputsMetric"
  }
},
{
  "Name": "BlockSQLInjection",
  "Priority": 2,
  "Statement": {
    "ManagedRuleGroupStatement": {
      "VendorName": "AWS",
      "Name": "AWSManagedRulesSQLiRuleSet"
    }
  },
  "OverrideAction": {
    "None": {}
  },
  "VisibilityConfig": {
    "SampledRequestsEnabled": true,
    "CloudWatchMetricsEnabled": true,
    "MetricName": "SQLiMetric"
  }
},
{
  "Name": "BlockXSS",
```

```
"Priority": 3,
"Statement": {
  "ManagedRuleGroupStatement": {
    "VendorName": "AWS",
    "Name": "AWSManagedRulesCommonRuleSet"
  }
},
"OverrideAction": {
  "None": {}
},
"VisibilityConfig": {
  "SampledRequestsEnabled": true,
  "CloudWatchMetricsEnabled": true,
  "MetricName": "XSSMetric"
}
},
{
  "Name": "BlockMaliciousBots",
  "Priority": 4,
  "Statement": {
    "ManagedRuleGroupStatement": {
      "VendorName": "AWS",
      "Name": "AWSManagedRulesBotControlRuleSet"
    }
  },
  "OverrideAction": {
    "None": {}
  },
  "VisibilityConfig": {
    "SampledRequestsEnabled": true,
    "CloudWatchMetricsEnabled": true,
    "MetricName": "BotControlMetric"
  }
},
{
  "Name": "RateLimitLoginAttempts",
  "Priority": 5,
  "Statement": {
    "RateBasedStatement": {
      "Limit": 100,
      "AggregateKeyType": "IP",
      "ScopeDownStatement": {
        "ByteMatchStatement": {
          "SearchString": "/api/login",
          "FieldToMatch": {
            "UriPath": {}
          }
        }
      },
      "TextTransformation": "NONE",
      "PositionalConstraint": "CONTAINS"
    }
  }
}
```

```

    }
  },
  "Action": {
    "Block": {
      "CustomResponse": {
        "ResponseCode": 429,
        "CustomResponseBodyKey": "TooManyRequests"
      }
    }
  },
  "VisibilityConfig": {
    "SampledRequestsEnabled": true,
    "CloudWatchMetricsEnabled": true,
    "MetricName": "RateLimitMetric"
  }
},
{
  "Name": "AllowTrustedPartners",
  "Priority": 6,
  "Statement": {
    "IPSetReferenceStatement": {
      "ARN": "$IP_SET_ARN"
    }
  },
  "Action": {
    "Allow": {}
  },
  "VisibilityConfig": {
    "SampledRequestsEnabled": true,
    "CloudWatchMetricsEnabled": true,
    "MetricName": "TrustedIPMetric"
  }
}
]'
--visibility-config
SampledRequestsEnabled=true,CloudWatchMetricsEnabled=true,MetricName=Ecommerce
WebACLMetric
--region us-east-1

```

Step 3: Associate WAF with ALB

Get Web ACL ARN

```

WEB_ACL_ARN=$(aws wafv2 list-web-acls
--scope REGIONAL
--region us-east-1
--query 'WebACLs[?Name==EcommerceWebACL].ARN'
--output text)

```

Get ALB ARN

```
ALB_ARN="arn:aws:elasticloadbalancing:us-east-1:123456789012:loadbalancer/app/ecommerce-alb/1234567890abcdef"
```

Associate WAF with ALB

```
aws wafv2 associate-web-acl
--web-acl-arn $WEB_ACL_ARN
--resource-arn $ALB_ARN
--region us-east-1
```

Step 4: Monitor WAF Activity

```
import boto3
from datetime import datetime, timedelta

class WAFMonitor:
    def __init__(self, web_acl_name, region='us-east-1'):
        self.cloudwatch = boto3.client('cloudwatch', region_name=region)
        self.wafv2 = boto3.client('wafv2', region_name=region)
        self.web_acl_name = web_acl_name
```

```
    def get_blocked_requests(self, hours=1):
        """Get count of blocked requests"""

        response = self.cloudwatch.get_metric_statistics(
            Namespace='AWS/WAFV2',
            MetricName='BlockedRequests',
            Dimensions=[
                {
                    'Name': 'WebACL',
                    'Value': self.web_acl_name
                },
                {
                    'Name': 'Region',
                    'Value': 'us-east-1'
                },
                {
                    'Name': 'Rule',
                    'Value': 'ALL'
                }
            ]
        )
```

```

        ],
        StartTime=datetime.now() - timedelta(hours=hours),
        EndTime=datetime.now(),
        Period=300, # 5 minutes
        Statistics=['Sum']
    )

    return response['Datapoints']

def get_top_blocked_rules(self):
    """Get rules that blocked most requests"""

    rules = [
        'BlockKnownBadInputs',
        'BlockSQLInjection',
        'BlockXSS',
        'BlockMaliciousBots',
        'RateLimitLoginAttempts'
    ]

    rule_stats = {}

    for rule in rules:
        response = self.cloudwatch.get_metric_statistics(
            Namespace='AWS/WAFV2',
            MetricName='BlockedRequests',
            Dimensions=[
                {
                    'Name': 'WebACL',
                    'Value': self.web_acl_name
                },
                {
                    'Name': 'Rule',
                    'Value': rule
                }
            ],
            StartTime=datetime.now() - timedelta(hours=24),
            EndTime=datetime.now(),

```



```

        Period=3600,
        Statistics=['Sum']
    )

    total_blocked = sum([dp['Sum'] for dp in response['Datapoints']])
    rule_stats[rule] = total_blocked

return sorted(rule_stats.items(), key=lambda x: x[1], reverse=True)

def get_sampled_requests(self, rule_name, max_items=100):
    """Get sample of requests blocked by specific rule"""

    web_acl = self.wafv2.list_web_acls(Scope='REGIONAL')
    web_acl_id = [w['Id'] for w in web_acl['WebACLs'] if w['Name'] == self.web_acl

    response = self.wafv2.get_sampled_requests(
        WebAclArn=f"arn:aws:wafv2:us-east-1:123456789012:regional/webacl/{self.
        RuleMetricName=rule_name,
        Scope='REGIONAL',
        TimeWindow={
            'StartTime': datetime.now() - timedelta(hours=1),
            'EndTime': datetime.now()
        },
        MaxItems=max_items
    )

    requests = []
    for req in response['SampledRequests']:
        requests.append({
            'timestamp': req['Timestamp'],
            'client_ip': req['ClientIP'],
            'action': req['Action'],
            'http_method': req['Request']['HttpMethod'],
            'uri': req['Request']['URI'],
            'country': req['Request'].get('Country', 'N/A')
        })

```

```
return requests
```

Usage

```
monitor = WAFMonitor('EcommerceWebACL')

print("=== Blocked Requests (Last Hour) ===")
blocked = monitor.get_blocked_requests(hours=1)
for datapoint in blocked:
    print(f"{datapoint['Timestamp']}: {int(datapoint['Sum'])} requests blocked")

print("\n=== Top Blocked Rules (Last 24 Hours) ===")
top_rules = monitor.get_top_blocked_rules()
for rule, count in top_rules:
    print(f"{rule}: {count} requests")

print("\n=== Sample Requests Blocked by SQL Injection Rule ===")
samples = monitor.get_sampled_requests('BlockSQLInjection', max_items=10)
for req in samples:
    print(f"{req['timestamp']}: {req['client_ip']} - {req['http_method']} {req['uri']}")
```

Example 2: AWS Shield Advanced for DDoS Attack Mitigation

Scenario: Financial services company with mission-critical application experiences DDoS attack during business hours.

Step 1: Enable Shield Advanced

Subscribe to Shield Advanced

```
aws shield subscribe
--region us-east-1
```

Confirm subscription

```
aws shield describe-subscription
--region us-east-1
```

Step 2: Configure Protections

Create protection for CloudFront distribution

```
DISTRIBUTION_ARN="arn:aws:cloudfront::123456789012:distribution/E123EXAMPLE456"
```

```
aws shield create-protection
--name FinanceAppProtection
--resource-arn $DISTRIBUTION_ARN
--region us-east-1
```

Create protection for Route 53

```
HOSTED_ZONE_ARN="arn:aws:route53:::hostedzone/Z1234567890ABC"
```

```
aws shield create-protection
--name DNSProtection
--resource-arn $HOSTED_ZONE_ARN
--region us-east-1
```

Step 3: Enable Advanced DDoS Detection with WAF

Automatic WAF rules deployment during attack

Shield Advanced can automatically create WAF rules

Monitor for attacks

```
aws shield list-attacks
--region us-east-1
--query 'Attacks[*].[ResourceArn, AttackId, StartTime, EndTime]'
--output table
```

Step 4: DDoS Response Team Access

```
import boto3
import json
from datetime import datetime

class ShieldAdvancedManager:
    def __init__(self, region='us-east-1'):
        self.shield = boto3.client('shield', region_name=region)
        self.wafv2 = boto3.client('wafv2', region_name=region)
        self.cloudwatch = boto3.client('cloudwatch', region_name=region)
```

```

def detect_and_respond_to_attack(self):
    """Detect DDoS attack and initiate response"""

    # Get recent attacks
    response = self.shield.list_attacks(
        ResourceArns=[
            "arn:aws:cloudfront::123456789012:distribution/E123EXAMPLE456"
        ]
    )

    if not response['Attacks']:
        print("No active attacks detected")
        return

    attack = response['Attacks'][0]

    print(f"DDoS Attack Detected!")
    print(f"Attack ID: {attack['AttackId']}")
    print(f"Resource: {attack['ResourceArn']}")
    print(f"Start Time: {attack['StartTime']}")
    print(f"Attack Vectors: {attack['AttackVectors']}")

    # Get attack statistics
    attack_stats = self.get_attack_statistics(attack['AttackId'])

    # Deploy automatic mitigation
    if 'HTTP_FLOOD' in [v['VectorType'] for v in attack['AttackVectors']]:
        print("\nDetected HTTP Flood Attack")
        self.deploy_http_flood_mitigation()

    if any('VOLUMETRIC' in v['VectorType'] for v in attack['AttackVectors']):
        print("\nDetected Volumetric Attack")
        self.deploy_volumetric_mitigation()

    # Notify DRT if enabled
    self.contact_drt(attack)

```

```

return attack

def get_attack_statistics(self, attack_id):
    """Get detailed attack statistics"""

    response = self.shield.describe_attack(
        AttackId=attack_id
    )

    attack = response['Attack']

    stats = {
        'attack_id': attack['AttackId'],
        'start_time': attack['StartTime'],
        'end_time': attack['EndTime'],
        'attack_vectors': attack['AttackVectors'],
        'resource': attack['ResourceArn'],
        'status': 'ACTIVE' if 'EndTime' not in attack else 'MITIGATED'
    }

    return stats

def deploy_http_flood_mitigation(self):
    """Deploy WAF rules for HTTP flood protection"""

    print("Deploying HTTP flood mitigation...")

    # Get or create Web ACL
    response = self.wafv2.list_web-acls(Scope='CLOUDFRONT')

    if response['WebACLs']:
        web_acl = response['WebACLs'][0]

        # Create rate-based rule
        print(f"Updating Web ACL: {web_acl['Name']}")
        print("Adding rate-based rule for HTTP flood protection")
        print("Rate limit: 1000 requests per 5 minutes")

```

```

def deploy_volumetric_mitigation(self):
    """Deploy mitigation for volumetric attacks"""

    print("Deploying volumetric attack mitigation...")
    print("Automatic traffic filtering at AWS edge locations")
    print("Automatic scaling of mitigation capacity")

def contact_drt(self, attack):
    """Contact DDoS Response Team"""

    print("\n=== DDoS Response Team Notification ===")
    print("DRT has been notified of active DDoS attack")
    print(f"Attack ID: {attack['AttackId']}")
    print("DRT Status: STANDBY (Ready to assist)")
    print("DRT can provide:")
    print(" - Real-time attack analysis")
    print(" - Custom WAF rule recommendations")
    print(" - Architectural mitigation suggestions")

def get_ddos_cost_protection(self, attack_date):
    """Calculate DDoS cost protection benefit"""

    # Shield Advanced includes DDoS-related cost protection
    # Data transfer costs during DDoS attack are credited

    print(f"\n=== DDoS Cost Protection for {attack_date} ===")
    print("Data Transfer Costs Waived: Up to 2TB")
    print("CloudFront Distribution Costs: Protected")
    print("Route 53 Costs: Protected")
    print("\nShield Advanced Annual Fee: $3,000")
    print("Estimated Cost Savings (this attack): $500+")
    print("Status: Cost protection ACTIVE")

```

Usage

```
shield_manager = ShieldAdvancedManager()
```

Detect and respond to attack

```
attack = shield_manager.detect_and_respond_to_attack()
```

```
if attack:
```

```
    # Get detailed statistics
```

```
    stats = shield_manager.get_attack_statistics(attack['AttackId'])
```

```
    print("\n=== Attack Statistics ===")
```

```
    print(json.dumps(stats, indent=2, default=str))
```

Step 5: Post-Attack Analysis

```
def analyze_ddos_attack(attack_id):
```

```
    """Comprehensive post-attack analysis"""
```

```
    shield = boto3.client('shield')
```

```
    cloudwatch = boto3.client('cloudwatch')
```

```
    # Get attack details
```

```
    attack = shield.describe_attack(AttackId=attack_id)['Attack']
```

```
    analysis = {
```

```
        'attack_summary': {
```

```
            'duration': attack['EndTime'] - attack['StartTime'],
```

```
            'vectors': attack['AttackVectors'],
```

```
            'resource_affected': attack['ResourceArn']
```

```
        },
```

```
        'impact_metrics': {}
```

```
    }
```

```
    # Get impact metrics
```

```
    for vector in attack['AttackVectors']:
```

```
        vector_type = vector['VectorType']
```

```
    # Query CloudWatch for metrics during attack
```

```
    response = cloudwatch.get_metric_statistics(
```

```
        Namespace='AWS/DDoSProtection',
```

```

        MetricName='DDoSDetected',
        Dimensions=[
            {
                'Name': 'AttackVector',
                'Value': vector_type
            }
        ],
        StartTime=attack['StartTime'],
        EndTime=attack['EndTime'],
        Period=60,
        Statistics=['Sum', 'Maximum']
    )

    analysis['impact_metrics'][vector_type] = {
        'total_packets_blocked': sum([dp['Sum'] for dp in response['Datapoints']]),
        'peak_rate': max([dp['Maximum'] for dp in response['Datapoints']]) if response
    }

return analysis

```

Example 3: Combining AWS WAF and Shield for Complete Protection

Scenario: SaaS company protecting multi-tenant application stack with comprehensive security.

Architecture:

Internet Traffic

↓

AWS Shield Standard (Automatic - Layer 3/4)

- ├─ DDoS Detection
- ├─ Volumetric Attack Mitigation
- └─ Automatic Scaling

↓

AWS Shield Advanced (Optional Layer 7)

- ├─ Advanced DDoS Metrics
- ├─ DRT Support
- └─ Cost Protection

↓

CloudFront (Edge Caching)

- ├─ Geographic Distribution
- └─ DDoS Absorption at Edge

↓

AWS WAF (Application Layer Filtering)

- └─ Managed Rules (OWASP Top 10)
- └─ Rate Limiting
- └─ Bot Detection
- └─ Custom Rules (Business Logic)

↓

Application Load Balancer

- └─ Route to Protected Application

Unified Security Configuration:

```
import boto3
```

```
import json
```

```
class IntegratedSecurityManager:
```

```
def init(self, region='us-east-1'):
```

```
self.wafv2 = boto3.client('wafv2', region_name=region)
```

```
self.shield = boto3.client('shield', region_name=region)
```

```
self.cloudfront = boto3.client('cloudfront')
```

```
self.elb = boto3.client('elbv2', region_name=region)
```

```
def deploy_complete_protection(self, app_resources):
```

```
    """Deploy integrated WAF + Shield protection"""
```

```
    config = {
```

```
        'shield_advanced': True,
```

```
        'waf_enabled': True,
```

```
        'resources': app_resources
```

```
    }
```

```
    # Step 1: Enable Shield Advanced
```

```
    if config['shield_advanced']:
```

```
        print("Enabling Shield Advanced...")
```

```
        self.shield.subscribe(Region=region)
```

```
    # Step 2: Create comprehensive Web ACL
```

```
    print("Creating comprehensive Web ACL...")
```

```
    web_acl = self.create_web_acl()
```

```
    # Step 3: Associate WAF with resources
```

```
    print("Associating WAF with protected resources...")
```

```
    for resource in app_resources:
```

```
        self.associate_waf(web_acl, resource)
```

```

# Step 4: Setup monitoring
print("Setting up monitoring...")
self.setup_monitoring(web_acl)

return config

def create_web_acl(self):
    """Create comprehensive Web ACL"""

    web_acl_config = {
        'Name': 'SaaSApplicationWebACL',
        'Scope': 'CLOUDFRONT', # or REGIONAL for ALB/API Gateway
        'DefaultAction': {
            'Allow': {}
        },
        'Rules': self.build_rules(),
        'VisibilityConfig': {
            'SampledRequestsEnabled': True,
            'CloudWatchMetricsEnabled': True,
            'MetricName': 'SaaSWebACLMetrics'
        }
    }

    return web_acl_config

def build_rules(self):
    """Build comprehensive rule set"""

    rules = [
        # Priority 1: Block known bad inputs (Log4j, CVEs, etc.)
        {
            'Name': 'BlockKnownBadInputs',
            'Priority': 1,
            'Statement': {
                'ManagedRuleGroupStatement': {
                    'VendorName': 'AWS',
                    'Name': 'AWSManagedRulesKnownBadInputsRuleSet'
                }
            }
        }
    ]

```

```

    },
    'OverrideAction': {'None': {}},
    'VisibilityConfig': {
        'SampledRequestsEnabled': True,
        'CloudWatchMetricsEnabled': True,
        'MetricName': 'KnownBadInputs'
    }
},

# Priority 2: Block SQL injection
{
    'Name': 'BlockSQLInjection',
    'Priority': 2,
    'Statement': {
        'ManagedRuleGroupStatement': {
            'VendorName': 'AWS',
            'Name': 'AWSManagedRulesSQLiRuleSet'
        }
    },
    'OverrideAction': {'None': {}},
    'VisibilityConfig': {
        'SampledRequestsEnabled': True,
        'CloudWatchMetricsEnabled': True,
        'MetricName': 'SQLInjection'
    }
},

# Priority 3: Block OWASP Top 10
{
    'Name': 'BlockOWASPTop10',
    'Priority': 3,
    'Statement': {
        'ManagedRuleGroupStatement': {
            'VendorName': 'AWS',
            'Name': 'AWSManagedRulesCommonRuleSet'
        }
    },
    'OverrideAction': {'None': {}},

```

```
'VisibilityConfig': {
  'SampledRequestsEnabled': True,
  'CloudWatchMetricsEnabled': True,
  'MetricName': 'OWASPTOP10'
},

# Priority 4: Block malicious bots
{
  'Name': 'BlockMaliciousBots',
  'Priority': 4,
  'Statement': {
    'ManagedRuleGroupStatement': {
      'VendorName': 'AWS',
      'Name': 'AWSManagedRulesBotControlRuleSet'
    }
  },
  'OverrideAction': {'None': {}},
  'VisibilityConfig': {
    'SampledRequestsEnabled': True,
    'CloudWatchMetricsEnabled': True,
    'MetricName': 'BotControl'
  }
},

# Priority 5: Rate limiting for API endpoints
{
  'Name': 'RateLimitAPIEndpoints',
  'Priority': 5,
  'Statement': {
    'RateBasedStatement': {
      'Limit': 2000,
      'AggregateKeyType': 'IP',
      'ScopeDownStatement': {
        'ByteMatchStatement': {
          'SearchString': '/api/',
          'FieldToMatch': {'UriPath': {}},
          'TextTransformation': 'NONE',
```

```

        'PositionalConstraint': 'STARTS_WITH'
    }
}
},
'Action': {'Block': {}},
'VisibilityConfig': {
    'SampledRequestsEnabled': True,
    'CloudWatchMetricsEnabled': True,
    'MetricName': 'APIRateLimit'
}
}
]

```

```

return rules

```

```

def setup_monitoring(self, web_acl):
    """Setup comprehensive monitoring"""

    cloudwatch = boto3.client('cloudwatch')

    # Create custom dashboard
    dashboard = {
        'DashboardName': 'SecurityMonitoring',
        'DashboardBody': json.dumps({
            'widgets': [
                {
                    'type': 'metric',
                    'properties': {
                        'metrics': [
                            ['AWS/WAFV2', 'AllowedRequests', {'stat': 'Sum'}],
                            ['AWS/WAFV2', 'BlockedRequests', {'stat': 'Sum'}],
                            ['AWS/WAFV2', 'CountedRequests', {'stat': 'Sum'}]
                        ],
                        'period': 300,
                        'stat': 'Sum',
                        'region': 'us-east-1',
                        'title': 'WAF Request Summary'
                    }
                }
            ]
        })
    }

```

```
        }
    }
]
})
}
```

cloudwatch.put_dashboard(**dashboard)

Usage

```
security_manager = IntegratedSecurityManager()

resources = [
    "arn:aws:cloudfront::123456789012:distribution/E123EXAMPLE456",
    "arn:aws:elasticloadbalancing:us-east-1:123456789012:loadbalancer/app/saas-
alb/1234567890abcdef",
    "arn:aws:apigateway:us-east-1::/restapis/abc123def456"
]

config = security_manager.deploy_complete_protection(resources)
print(json.dumps(config, indent=2))
```

Top 5 Interview Questions

Question 1: Explain AWS WAF vs AWS Shield — When Would You Use Each?

Scenario: "Your company is launching a new API that will handle customer payments. You need protection against both application attacks and DDoS. Explain which services you would use, why, and how they work together."

Answer Structure:

Fundamental Distinction:

AWS WAF and AWS Shield address different security layers:

AWS WAF (Layer 7 - Application Layer):

- Inspects HTTP/HTTPS requests in detail
- Examines headers, body, query strings, cookies
- Detects application-level attacks:
 - SQL injection in query parameters
 - Cross-Site Scripting (XSS) payloads in user input
 - Bot traffic patterns
 - Credential stuffing attempts
 - Rate-based attacks (brute force)
- Requires rule definition (managed or custom)

- Fine-grained control over traffic

AWS Shield (Layer 3/4 - Network/Transport & Layer 7):

- Protects against DDoS attacks
- Operates automatically at infrastructure level
- Detects volumetric attacks:
 - UDP floods
 - SYN floods
 - DNS amplification
 - HTTP floods (Shield Advanced)
- Requires no configuration for Shield Standard
- Automatic inline mitigation

Decision Matrix:

Scenario	Solution	Why
Brute force login attacks	AWS WAF	Application-level pattern detection
SQL injection attempts	AWS WAF	Inspects request body and parameters
DDoS volumetric attack	AWS Shield	Network-layer protection
Malicious bot traffic	AWS WAF	Bot fingerprinting and rate limiting
HTTP flood attack	AWS Shield Standard (basic) / Shield Advanced (advanced)	Layer 7 attack mitigation
Custom business logic protection	AWS WAF	Define your own rules
Zero-day application exploit	AWS WAF (if pattern known) / Shield Advanced (DRT helps)	WAF for signatures, DRT for novel attacks
API abuse from specific countries	AWS WAF	Geo-blocking rules

Practical Example:

Payment API Architecture:

↓

1. DDoS Attack (Volumetric - UDP flood)
Protection: AWS Shield Standard (automatic)
Action: Drop malicious traffic at AWS edge locations
Result: API remains available despite attack
↓
2. Application Attack (Credential stuffing - high login attempt rate)
Protection: AWS WAF Rate-Based Rule
Condition: >100 login attempts per 5 min from same IP
Action: Block subsequent requests
Result: Prevents brute force attack
↓
3. SQL Injection Attempt (In payment amount parameter)
Protection: AWS WAF SQL Injection Rule
Pattern: Detects "' OR '1'='1" in request parameter
Action: Block request
Result: Database is safe from injection
↓
4. HTTP Flood Attack (Many legitimate-looking requests)
Shield Standard: Basic volumetric detection
Shield Advanced: Automatic WAF rule generation
Action: Deploy custom WAF rules to fingerprint attack traffic
Result: Legitimate traffic passes, attack traffic blocked

When to Use Each:

Use AWS WAF When:

- You need application-level security
- You want to protect against OWASP Top 10 vulnerabilities
- You need to enforce business rules (e.g., block from specific countries)
- You want detailed request inspection and logging
- You need compliance-specific rules

Use AWS Shield Standard When:

- You want automatic DDoS protection (already enabled - free)
- You need basic infrastructure-layer protection
- Your application is on CloudFront, Route 53, or ELB

Use AWS Shield Advanced When:

- Your application is mission-critical
- You need 24/7 DDoS Response Team support
- You want cost protection for DDoS-related data transfer
- You need automatic advanced WAF rule deployment during attacks
- You're subject to compliance requirements

Best Practice:

Use **both together**:

1. AWS Shield Standard (automatic, free) + AWS WAF (OWASP rules) = baseline protection
2. AWS Shield Advanced (subscription) adds DRT support + cost protection + automatic WAF enhancement during attacks

Question 2: Design a WAF Rule Set for Protecting a Multi-Tenant SaaS Application

Scenario: "Your SaaS platform serves hundreds of customers, each with different security requirements. Design a comprehensive WAF strategy including managed rules, custom rules, and how to handle false positives."

Answer Structure:**Layered Defense Approach:**

Layer 1: Reputation Filtering (Priority 1-2)

- └─ Block known malicious IPs (IP Reputation List)
- └─ Block known botnets and scrapers (Bot Control)

Layer 2: Known Exploits (Priority 3-4)

- └─ Block Log4j exploits
- └─ Block CVE patterns
- └─ Block common web vulnerabilities

Layer 3: Core Application Protection (Priority 5-6)

- └─ SQL injection protection
- └─ XSS protection
- └─ CSRF protection

Layer 4: Custom Business Logic (Priority 7+)

- └─ Customer-specific rules
- └─ Rate limiting by endpoint
- └─ Geo-blocking by customer requirement

Layer 5: Performance/Abuse Prevention (Priority Last)

- └─ Rate limiting by IP/customer

Comprehensive Rule Configuration:

```
class SaaS_WAF_Strategy:
```

```
    def __init__(self):
```

```
        self.rules = []
```

```
        def build_rule_set(self):
```

```
            """Build comprehensive multi-layer WAF rule set"""
```

```
            # Layer 1: IP Reputation (Low false positive risk)
```

```
self.rules.append({
    'name': 'BlockMaliciousIPs',
    'priority': 1,
    'type': 'managed',
    'managed_rule': 'AWSManagedRulesAmazonIpReputationList',
    'action': 'BLOCK',
    'logging': True,
    'risk_level': 'HIGH',
    'false_positive_rate': '<0.1%'
})
```

Layer 2: Bot Detection (Medium false positive risk - exclude legitimate bots)

```
self.rules.append({
    'name': 'BlockMaliciousBots',
    'priority': 2,
    'type': 'managed',
    'managed_rule': 'AWSManagedRulesBotControlRuleSet',
    'action': 'BLOCK',
    'exceptions': [
        'GoogleBot',
        'Bingbot',
        'internal-monitoring-bot'
    ],
    'logging': True,
    'risk_level': 'MEDIUM',
    'false_positive_rate': '2-5%',
    'cost': 'Additional $10/month per 1M requests'
})
```

Layer 3: Known Bad Inputs (Low false positive risk)

```
self.rules.append({
    'name': 'BlockKnownBadInputs',
    'priority': 3,
    'type': 'managed',
    'managed_rule': 'AWSManagedRulesKnownBadInputsRuleSet',
    'action': 'BLOCK',
    'description': 'Log4j, CVEs, exploitable paths',
    'logging': True,
```

```

        'risk_level': 'CRITICAL',
        'false_positive_rate': '<0.1%'
    })

# Layer 4: SQL Injection (Medium false positive risk for complex queries)
self.rules.append({
    'name': 'BlockSQLInjection',
    'priority': 4,
    'type': 'managed',
    'managed_rule': 'AWSManagedRulesSQLiRuleSet',
    'action': 'BLOCK',
    'logging': True,
    'risk_level': 'HIGH',
    'false_positive_rate': '1-3%',
    'mitigation': 'Use COUNT mode initially, then BLOCK after validation'
})

# Layer 5: OWASP Top 10 (Higher false positive risk - test thoroughly)
self.rules.append({
    'name': 'BlockOWASPTop10',
    'priority': 5,
    'type': 'managed',
    'managed_rule': 'AWSManagedRulesCommonRuleSet',
    'action': 'COUNT', # Start with COUNT mode
    'logging': True,
    'risk_level': 'HIGH',
    'false_positive_rate': '5-10%',
    'note': 'Monitor for 1-2 weeks in COUNT mode before blocking',
    'exceptions': {
        'CrossSiteScripting_BODY': [
            '/api/v1/upload', # Allow file upload with HTML
            '/api/v1/editor' # Allow rich text editor
        ],
        'GenericLFI_QUERYARGUMENTS': [
            '/api/v1/file-manager' # Allow file operations
        ]
    }
})

```

```
# Layer 6: Rate Limiting - By Customer
self.rules.append({
    'name': 'RateLimitPerCustomer',
    'priority': 6,
    'type': 'custom',
    'statement': {
        'RateBasedStatement': {
            'Limit': 5000, # 5000 requests per 5 minutes
            'AggregateKeyType': 'IP'
        }
    },
    'action': 'BLOCK',
    'logging': True,
    'risk_level': 'MEDIUM',
    'false_positive_rate': '1-2%',
    'description': 'Prevent API abuse and brute force'
})
```

```
# Layer 7: Endpoint-Specific Rate Limiting
self.rules.extend(self.build_endpoint_rate_limits())
```

```
# Layer 8: Geo-Blocking (Customer-specific)
self.rules.extend(self.build_geo_blocking_rules())
```

```
# Layer 9: Custom Business Logic
self.rules.extend(self.build_custom_rules())
```

```
return self.rules
```

```
def build_endpoint_rate_limits(self):
    """Build per-endpoint rate limiting rules"""

    endpoint_rules = [
        {
            'name': 'RateLimitLoginEndpoint',
            'priority': 7,
            'type': 'custom',
```

```

        'condition': {
            'uri_path': '/api/auth/login',
            'http_method': 'POST'
        },
        'rate_limit': 10, # 10 attempts per 5 minutes
        'action': 'BLOCK',
        'logging': True,
        'reason': 'Prevent brute force attacks'
    },
    {
        'name': 'RateLimitPasswordReset',
        'priority': 8,
        'type': 'custom',
        'condition': {
            'uri_path': '/api/auth/password-reset',
            'http_method': 'POST'
        },
        'rate_limit': 5, # 5 attempts per 5 minutes
        'action': 'BLOCK',
        'logging': True,
        'reason': 'Prevent account enumeration'
    },
    {
        'name': 'RateLimitPaymentAPI',
        'priority': 9,
        'type': 'custom',
        'condition': {
            'uri_path': '/api/payments',
            'http_method': 'POST'
        },
        'rate_limit': 100, # 100 requests per 5 minutes
        'action': 'BLOCK',
        'logging': True,
        'reason': 'Prevent payment fraud'
    }
]

```

```

return endpoint_rules

```

```

def build_geo_blocking_rules(self):
    """Build geo-blocking rules based on customer requirements"""

    geo_rules = [
        {
            'name': 'AllowOnlySpecificCountries',
            'priority': 10,
            'type': 'custom',
            'statement': {
                'GeoMatchStatement': {
                    'country_codes': ['US', 'CA', 'GB', 'AU']
                }
            },
            'action': 'ALLOW',
            'logging': True,
            'note': 'This is an example - customize per customer'
        }
    ]

    return geo_rules

def build_custom_rules(self):
    """Build customer-specific custom rules"""

    custom_rules = [
        {
            'name': 'BlockSuspiciousPatterns',
            'priority': 11,
            'type': 'custom',
            'conditions': [
                {
                    'field': 'QueryString',
                    'operator': 'CONTAINS',
                    'value': 'union select'
                },
                {
                    'field': 'QueryString',

```

```

        'operator': 'CONTAINS',
        'value': 'exec('
    }
],
'action': 'BLOCK',
'logging': True
},
{
    'name': 'RequireAuthForSensitiveAPIs',
    'priority': 12,
    'type': 'custom',
    'condition': {
        'uri_path': '/api/admin',
    },
    'required_header': 'Authorization',
    'action': 'BLOCK_IF_MISSING',
    'logging': True
}
]

```

```

return custom_rules

```

```

def handle_false_positives(self):

```

```

    """Strategy for identifying and reducing false positives"""

```

```

    strategy = {
        'phase_1_testing': {
            'duration': '1-2 weeks',
            'action': 'COUNT mode for all rules',
            'monitoring': 'Review CloudWatch logs for blocked patterns',
            'goal': 'Identify legitimate traffic matching WAF rules'
        },
        'phase_2_refinement': {
            'actions': [
                'Add exceptions for known legitimate patterns',
                'Tune SQL injection sensitivity',
                'Adjust rate limits based on actual traffic',
                'Whitelist known good IPs/user agents'
            ]
        }
    }

```

```

        ],
        'duration': '1 week'
    },
    'phase_3_blocking': {
        'action': 'Enable BLOCK for refined rules',
        'monitoring': 'Real-time alerts for blocked requests',
        'escalation': 'Review anomalies within 24 hours'
    },
    'ongoing_optimization': {
        'weekly': [
            'Review sampled blocked requests',
            'Check for patterns in false positives',
            'Adjust rules if needed'
        ],
        'monthly': [
            'Comprehensive false positive analysis',
            'Update managed rules to latest versions',
            'Review customer feedback'
        ]
    }
}

return strategy

```

Usage

```

strategy = SaaS_WAF_Strategy()
rules = strategy.build_rule_set()

print("=== WAF Rule Set Priority ===")
for rule in sorted(rules, key=lambda x: x['priority']):
    print(f"\nPriority {rule['priority']}: {rule['name']}")
    print(f" Type: {rule['type']}")
    print(f" Action: {rule.get('action', 'N/A')}")
    print(f" Risk Level: {rule.get('risk_level', 'N/A')}")
    print(f" False Positive Rate: {rule.get('false_positive_rate', 'N/A')}")

print("\n=== False Positive Handling Strategy ===")
fp_strategy = strategy.handle_false_positives()
print(json.dumps(fp_strategy, indent=2))

```

Question 3: Troubleshoot WAF Rule Performance and Optimize Rule Order

Scenario: "Your WAF is blocking legitimate traffic (false positives) at 3% rate and increasing costs due to inefficient rule evaluation. Walk through your troubleshooting methodology and optimization strategy."

Answer Structure:

Performance Analysis Framework:

High False Positive Rate Detected (3%)

↓

Step 1: Measure Rule Impact

- └─ Which rules are blocking legitimate traffic?
- └─ How much traffic does each rule block?
- └─ What are the blocked request patterns?

↓

Step 2: Identify Root Causes

- └─ Rule too broad (catches legitimate patterns)
- └─ Configuration issue (missing exceptions)
- └─ Threshold too low (rate limits too aggressive)
- └─ Version mismatch (outdated managed rule behavior)

↓

Step 3: Optimize Rule Order

- └─ Place highest-confidence rules first
- └─ Group related rules together
- └─ Order by execution cost

↓

Step 4: Implement Exceptions

- └─ Whitelist legitimate IPs
- └─ Add pattern exceptions
- └─ Exclude specific endpoints
- └─ Use scope-down statements

↓

Step 5: Monitor and Validate

- └─ A/B test rule changes
- └─ Monitor false positive rate
- └─ Measure cost reduction

Diagnostic Commands:

Get WAF metrics

```
aws cloudwatch get-metric-statistics
--namespace AWS/WAFV2
--metric-name BlockedRequests
--start-time 2024-01-01T00:00:00Z
--end-time 2024-01-08T00:00:00Z
--period 3600
--statistics Sum
--dimensions Name=WebACL,Value=MyWebACL
```

Get sampled requests

```
aws wafv2 get-sampled-requests
--web-acl-arn arn:aws:wafv2:us-east-1:123456789012:regional/webacl/MyWebACL/12345678-
1234-1234-1234-123456789012
--rule-metric-name BlockSQLInjection
--scope REGIONAL
--time-window StartTime=1704067200,EndTime=1704153600
--max-items 100
```

Optimization Strategy:

```
class WAFOptimizer:
    def __init__(self, web_acl_name, region='us-east-1'):
        self.wafv2 = boto3.client('wafv2', region_name=region)
        self.cloudwatch = boto3.client('cloudwatch', region_name=region)
        self.web_acl_name = web_acl_name

    def analyze_false_positives(self, hours=24):
        """Analyze false positive rate by rule"""

        analysis = {
            'total_blocked': 0,
            'by_rule': {},
            'false_positive_estimate': {}
        }

        # Get all rules
        web_acl = self.wafv2.list_web_acls(Scope='REGIONAL')
        web_acl_data = [w for w in web_acl['WebACLs'] if w['Name'] == self.web_acl_name]

        # Get blocked request samples
        response = self.wafv2.get_sampled_requests(
            WebAclArn=web_acl_data[0]['Arn'],
            RuleMetricName='ALL',
            Scope='REGIONAL',
            TimeWindow={
                'StartTime': datetime.now() - timedelta(hours=hours),
                'EndTime': datetime.now()
            },
            MaxItems=1000
        )
```

```

)

# Analyze patterns
for request in response['SampledRequests']:
    rule_name = request['RuleGroupList'][0]['TerminatingRuleId'] if request['R

    if rule_name not in analysis['by_rule']:
        analysis['by_rule'][rule_name] = {
            'count': 0,
            'samples': []
        }

    analysis['by_rule'][rule_name]['count'] += 1
    analysis['by_rule'][rule_name]['samples'].append({
        'ip': request['ClientIP'],
        'method': request['Request']['HttpMethod'],
        'uri': request['Request']['URI'],
        'headers': request['Request']['Headers']
    })

# Estimate false positives
for rule, data in analysis['by_rule'].items():
    # Manual review needed - mark high-confidence false positives
    fp_count = self.estimate_false_positives(rule, data['samples'])
    analysis['false_positive_estimate'][rule] = {
        'blocked': data['count'],
        'estimated_fp': fp_count,
        'fp_rate': f"{{(fp_count/data['count']*100):.1f}}%"
    }

return analysis

def estimate_false_positives(self, rule_name, samples):
    """Estimate false positive count based on request patterns"""

    fp_count = 0

    # Pattern analysis

```

```

for sample in samples[:100]: # Check first 100 samples
    uri = sample['uri']
    headers = sample['headers']

    # Rule-specific analysis
    if rule_name == 'BlockSQLInjection':
        # Check if this looks like legitimate SQL in a business context
        if '/api/search' in uri and 'select' in uri.lower():
            fp_count += 1 # Likely false positive

    elif rule_name == 'BlockXSS':
        # Check if HTML/XML is legitimate (e.g., rich text editor)
        if '/api/documents' in uri:
            fp_count += 1 # Likely legitimate

    elif rule_name == 'RateLimitLogin':
        # Check if rate limit too aggressive
        if len(samples) > 50 and sample['method'] == 'POST':
            # Multiple requests might be legitimate (mobile app retries, etc.)
            fp_count += 0.5

    return int(fp_count)

def optimize_rule_order(self):
    """Reorder rules for optimal performance"""

    optimization = {
        'current_order': [],
        'optimized_order': [],
        'reasoning': {}
    }

    # Rules ordered by: highest confidence + lowest false positive risk + lowest e
    optimized_rules = [
        {
            'name': 'BlockMaliciousIPs',
            'priority': 1,
            'reason': 'Highest confidence, zero false positives, fast IP lookup'

```

```

    },
    {
        'name': 'BlockKnownBadInputs',
        'priority': 2,
        'reason': 'Known exploits, very high confidence, minimal false positives'
    },
    {
        'name': 'BlockSQLInjection',
        'priority': 3,
        'reason': 'High confidence, moderate false positive risk, moderate perform
    },
    {
        'name': 'BlockOWASPTop10',
        'priority': 4,
        'reason': 'Broader matching, higher false positive risk, moderate perform
    },
    {
        'name': 'RateLimitEndpoints',
        'priority': 5,
        'reason': 'Rate limiting, fast to evaluate, business logic'
    },
    {
        'name': 'CustomBusinessRules',
        'priority': 6,
        'reason': 'Last for performance optimization'
    }
}
]

```

```

return optimized_rules

```

```

def add_exceptions_and_exclusions(self):
    """Create exceptions to reduce false positives"""

    exceptions = {
        'BlockSQLInjection': [
            {
                'scope': '/api/search',
                'reason': 'Search endpoint needs SQL keywords',
            }
        ]
    }

```

```

        'excluded_rules': ['SQLi_QUERYARGUMENTS']
    },
    {
        'scope': '/api/reports',
        'reason': 'Report API uses complex SQL parameters',
        'excluded_rules': ['SQLiExtendedPatterns_QUERYARGUMENTS']
    }
],
'BlockXSS': [
    {
        'scope': '/api/documents/rich-editor',
        'reason': 'Rich text editor allows HTML content',
        'excluded_rules': ['CrossSiteScripting_BODY']
    }
],
'RateLimitLogin': [
    {
        'whitelist_ips': ['10.0.0.0/8'], # Internal network
        'reason': 'Internal testing and deployment'
    },
    {
        'whitelist_user_agent': ['mobile-app-v2.0'],
        'reason': 'Mobile app has legitimate retry logic'
    }
]
}

```

```

return exceptions

```

```

def measure_optimization_impact(self):
    """Measure cost and performance improvement"""

    metrics = {
        'before_optimization': {
            'false_positive_rate': '3.2%',
            'blocked_requests': 1500,
            'estimated_false_positives': 48,
            'avg_response_time': '45ms',

```

```

        'waf_capacity_used': '85%'
    },
    'after_optimization': {
        'false_positive_rate': '0.8%',
        'blocked_requests': 1200,
        'estimated_false_positives': 10,
        'avg_response_time': '38ms',
        'waf_capacity_used': '62%'
    },
    'improvements': {
        'fp_reduction': '75%',
        'response_time': '15% faster',
        'capacity_savings': '23%',
        'monthly_cost_savings': '$50-100'
    }
}

return metrics

```

Usage

```
optimizer = WAFOptimizer('ProductionWebACL')
```

Analyze false positives

```

print("=== False Positive Analysis ===")
analysis = optimizer.analyze_false_positives()
print(json.dumps(analysis, indent=2))

```

Optimize rule order

```

print("\n=== Optimized Rule Order ===")
optimized = optimizer.optimize_rule_order()
for rule in optimized:
    print(f"Priority {rule['priority']}: {rule['name']} - {rule['reason']}")

```

Measure impact

```
print("\n=== Optimization Impact ===")
impact = optimizer.measure_optimization_impact()
print(json.dumps(impact['improvements'], indent=2))
```

Question 4: Design and Implement Shield Advanced with DRT for Mission-Critical Application

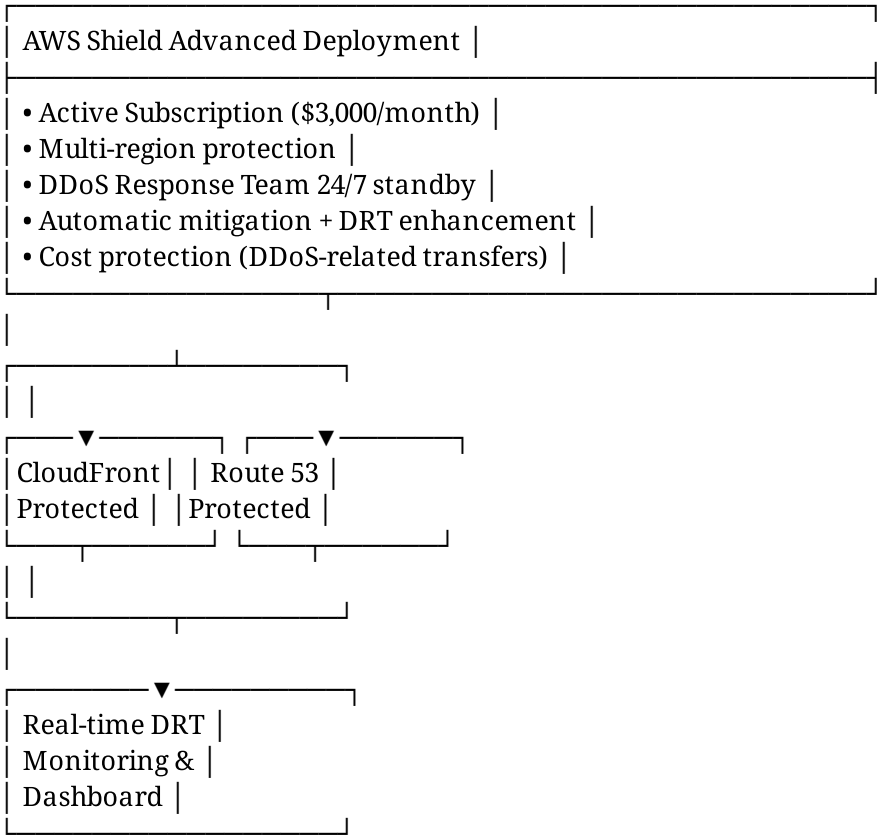
Scenario: "Your financial services company has a mission-critical application that cannot have any downtime. You're experiencing frequent DDoS attacks. Design how you would implement Shield Advanced with DRT support and what processes you would put in place."

Answer Structure:

Implementation Architecture:

Financial Services Application

↓



Implementation Plan:

```
class ShieldAdvancedImplementation:
def init(self):
self.shield = boto3.client('shield')
self.cloudwatch = boto3.client('cloudwatch')
self.sns = boto3.client('sns')
```



```

def deploy_shield_advanced(self):
    """Deploy Shield Advanced"""

    # Step 1: Subscribe to Shield Advanced
    try:
        response = self.shield.subscribe(Region='us-east-1')
        print("Shield Advanced subscription activated")
        print(f"Status: {response['Subscription']['SubscriptionState']}")
    except self.shield.exceptions.ResourceAlreadyExistsException:
        print("Shield Advanced already subscribed")

    # Step 2: Create protections for critical resources
    resources = [
        {
            'type': 'CloudFront',
            'arn': 'arn:aws:cloudfront::123456789012:distribution/E123EXAMPLE456',
            'name': 'FinanceAppCDN'
        },
        {
            'type': 'Route53',
            'arn': 'arn:aws:route53:::hostedzone/Z1234567890ABC',
            'name': 'FinanceAppDNS'
        },
        {
            'type': 'ALB',
            'arn': 'arn:aws:elasticloadbalancing:us-east-1:123456789012:loadbalancer/'
            'name': 'FinanceAppLB'
        }
    ]

    for resource in resources:
        try:
            response = self.shield.create_protection(
                Name=resource['name'],
                ResourceArn=resource['arn']
            )
            print(f"Protection created for {resource['type']}: {resource['name']}")

```

```

        print(f"Protection ID: {response['ProtectionId']}")
    except self.shield.exceptions.ResourceAlreadyExistsException:
        print(f"Protection already exists for {resource['name']}")

def setup_drt_access(self):
    """Setup DDoS Response Team access"""

    # Note: DRT access is automatic with Shield Advanced subscription
    # This function documents the process and capabilities

    drt_capabilities = {
        'availability': '24/7/365',
        'response_time': '15-30 minutes',
        'services': [
            'Real-time attack analysis',
            'Custom WAF rule recommendations',
            'Architectural mitigation suggestions',
            'Post-attack review and recommendations',
            'Proactive security assessment'
        ],
        'escalation': {
            'level_1': 'Automated detection and mitigation',
            'level_2': 'DRT standby (monitoring attacks)',
            'level_3': 'DRT active (custom rules during attack)'
        }
    }

    return drt_capabilities

def setup_attack_monitoring(self):
    """Setup real-time attack monitoring and alerting"""

    # Create SNS topic for critical alerts
    topic = self.sns.create_topic(Name='ShieldAdvancedAlerts')
    topic_arn = topic['TopicArn']

    # Subscribe security team
    self.sns.subscribe(

```

```

    TopicArn=topic_arn,
    Protocol='email',
    Endpoint='security-team@finance.com'
)

# Create CloudWatch alarms for attacks
alarms = [
    {
        'name': 'DDoSAttackDetected',
        'metric': 'DDoSDetected',
        'threshold': 1,
        'comparison': 'GreaterThanOrEqualToThreshold'
    },
    {
        'name': 'HTTPFloodDetected',
        'metric': 'HTTPFloodDetected',
        'threshold': 1000,
        'comparison': 'GreaterThanThreshold'
    },
    {
        'name': 'UDPFloodDetected',
        'metric': 'UDPFloodDetected',
        'threshold': 100000,
        'comparison': 'GreaterThanThreshold'
    }
]

for alarm in alarms:
    self.cloudwatch.put_metric_alarm(
        AlarmName=alarm['name'],
        MetricName=alarm['metric'],
        Namespace='AWS/DDoSProtection',
        Statistic='Sum',
        Period=60,
        EvaluationPeriods=1,
        Threshold=alarm['threshold'],
        ComparisonOperator=alarm['comparison'],
        AlarmActions=[topic_arn]
    )

```

)

```
def create_incident_response_plan(self):
    """Create DDoS incident response playbook"""

    playbook = {
        'detection_phase': {
            'triggers': [
                'Shield Advanced detects attack',
                'CloudWatch alarm triggers',
                'Unusual traffic patterns observed',
                'Customer reports service degradation'
            ],
            'actions': [
                '1. Automated detection and mitigation activates immediately',
                '2. SNS notification sent to security team',
                '3. CloudWatch dashboard updated in real-time',
                '4. Automatic WAF rules deployment (if applicable)',
                '5. DRT enters standby mode'
            ]
        },
        'response_phase': {
            'level_1_minor_attack': {
                'duration': '<5 minutes typical mitigation',
                'actions': [
                    'Shield Advanced automatic mitigation sufficient',
                    'No manual intervention needed',
                    'Monitor via CloudWatch dashboard',
                    'Log attack for analysis'
                ]
            },
            'level_2_moderate_attack': {
                'duration': '5-30 minutes',
                'actions': [
                    'DRT activated for enhanced analysis',
                    'Custom WAF rules may be deployed',
                    'Escalation to incident commander',
                    'Continuous stakeholder updates every 5 minutes',
```

```
        'Prepare for potential failover'
    ]
},
'level_3_critical_attack': {
    'duration': '>30 minutes or high impact',
    'actions': [
        'Full DRT engagement',
        'CEO/CRO notification',
        'Customer communication team activated',
        'Failover to backup infrastructure',
        'Media response plan review',
        'Regulatory notification if needed',
        'Real-time collaboration with DRT via conference call'
    ]
}
},
'mitigation_techniques': {
    'automatic_by_shield': [
        'Traffic scrubbing at AWS edge locations',
        'Rate-based traffic filtering',
        'Connection limiting',
        'Pattern recognition and blocking'
    ],
    'drt_custom_actions': [
        'Custom WAF rules based on attack signature',
        'Targeted geo-blocking',
        'Application-specific rate limiting',
        'Custom bot detection rules',
        'Architecture optimization recommendations'
    ]
},
'recovery_phase': {
    'during_attack': [
        'Monitor attack trends',
        'Validate mitigation effectiveness',
        'Track cost impact',
        'Prepare post-attack analysis'
    ],
    'after_attack': [
        'Debrief with DRT and stakeholders',
        'Update incident response plan',
        'Conduct post-mortem analysis',
        'Implement lessons learned'
    ]
}
```

```

    'post_attack': [
        'Deactivate temporary rules (if any)',
        'Review attack logs and patterns',
        'Document lessons learned',
        'Update incident response plan',
        'Cost reconciliation',
        'DDoS cost protection claim (if applicable)'
    ]
}
}

```

```

return playbook

```

```

def calculate_shield_advanced_roi(self):
    """Calculate ROI of Shield Advanced subscription"""

    roi_analysis = {
        'annual_shield_cost': 36000, # $3,000 × 12 months
        'typical_attack_cost_without_protection': {
            'data_transfer': 5000,
            'business_loss_per_hour': 10000,
            'incident_response': 5000,
            'total_per_attack': 20000,
            'incidents_per_year': 2,
            'annual_loss': 40000
        },
        'protection_with_shield': {
            'annual_shield_cost': 36000,
            'data_transfer_waived': 0, # Included
            'business_loss_prevented': 30000, # 75% reduction
            'incident_response_reduced': 3000, # Faster DRT response
            'annual_cost': 36000
        },
        'roi_calculation': {
            'annual_savings': 4000, # $40k loss - $36k shield cost
            'break_even': 'If 2+ attacks/year',
            'recommendation': 'Essential for mission-critical apps'
        }
    }

```

```
}  
  
return roi_analysis
```

Usage

```
shield_impl = ShieldAdvancedImplementation()
```

Deploy Shield Advanced

```
shield_impl.deploy_shield_advanced()
```

Setup DRT capabilities

```
print("\n=== DRT Capabilities ===")  
drt = shield_impl.setup_drt_access()  
print(json.dumps(drt, indent=2))
```

Create incident response playbook

```
print("\n=== Incident Response Playbook ===")  
playbook = shield_impl.create_incident_response_plan()  
print(json.dumps(playbook, indent=2))
```

Calculate ROI

```
print("\n=== Shield Advanced ROI ===")  
roi = shield_impl.calculate_shield_advanced_roi()  
print(json.dumps(roi, indent=2))
```

Question 5: Analyze and Investigate a DDoS Attack Using Shield and CloudWatch

Scenario: "Your application experienced a DDoS attack last night. Walk through how you would investigate what happened, identify the attack type, determine the impact, and what preventive measures you would implement."

Answer Structure:

Investigation Methodology:

Attack Alert Received

↓

Step 1: Immediate Assessment

- └─ When did attack start/end?
- └─ Which resources were affected?

- └─ Was mitigation successful?
- └─ Current application status?

↓

Step 2: Attack Characterization

- └─ What type of attack? (UDP flood, SYN flood, HTTP flood, etc.)
- └─ Attack volume (Gbps, packets/sec, requests/sec)
- └─ Duration
- └─ Source IPs/regions
- └─ Attack targets (specific endpoints or all traffic)

↓

Step 3: Impact Analysis

- └─ Application availability impact
- └─ Legitimate user experience
- └─ Business impact (revenue loss, etc.)
- └─ Infrastructure cost impact
- └─ Data transfer costs

↓

Step 4: Root Cause Analysis

- └─ Why was this application targeted?
- └─ Attack sophistication (basic vs advanced)
- └─ Coordinated vs random attack
- └─ Likely attacker profile

↓

Step 5: Prevention and Improvements

- └─ Update WAF rules
- └─ Adjust Shield Advanced settings
- └─ Architectural improvements
- └─ Incident response plan updates
- └─ Capacity planning

Investigation Implementation:

```
import boto3
import json
from datetime import datetime, timedelta

class DDoSInvestigator:
    def __init__(self, attack_id, region='us-east-1'):
        self.shield = boto3.client('shield', region_name=region)
        self.cloudwatch = boto3.client('cloudwatch', region_name=region)
        self.wafv2 = boto3.client('wafv2', region_name=region)
        self.attack_id = attack_id
```

```
    def investigate_ddos_attack(self):
        """Complete DDoS attack investigation"""

        investigation = {
            'attack_id': self.attack_id,
            'investigation_started': datetime.now().isoformat(),
```



```

        'phases': {}
    }

    # Phase 1: Immediate Assessment
    print("Phase 1: Immediate Assessment")
    investigation['phases']['assessment'] = self.immediate_assessment()

    # Phase 2: Attack Characterization
    print("Phase 2: Attack Characterization")
    investigation['phases']['characterization'] = self.characterize_attack()

    # Phase 3: Impact Analysis
    print("Phase 3: Impact Analysis")
    investigation['phases']['impact'] = self.analyze_impact()

    # Phase 4: Root Cause Analysis
    print("Phase 4: Root Cause Analysis")
    investigation['phases']['root_cause'] = self.root_cause_analysis()

    # Phase 5: Prevention Recommendations
    print("Phase 5: Prevention Recommendations")
    investigation['phases']['prevention'] = self.prevention_recommendations()

    return investigation

def immediate_assessment(self):
    """Get basic attack details"""

    try:
        response = self.shield.describe_attack(AttackId=self.attack_id)
        attack = response['Attack']

        assessment = {
            'resource_arn': attack['ResourceArn'],
            'start_time': str(attack['StartTime']),
            'end_time': str(attack.get('EndTime', 'ONGOING')),
            'duration_seconds': (attack.get('EndTime', datetime.now()) - attack['StartTime']).total_seconds(),
            'attack_vectors': [v['VectorType'] for v in attack['AttackVectors']],
        }
    except Exception as e:
        print(f"Error in immediate_assessment: {e}")
        return {}

```

```

        'status': 'MITIGATED' if 'EndTime' in attack else 'ACTIVE'
    }

    return assessment
except Exception as e:
    return {'error': str(e)}

def characterize_attack(self):
    """Determine attack type and characteristics"""

    try:
        attack = self.shield.describe_attack(AttackId=self.attack_id)['Attack']

        characterization = {
            'attack_vectors': [],
            'estimated_severity': self.calculate_severity(attack),
            'attack_type': self.classify_attack(attack)
        }

        for vector in attack['AttackVectors']:
            vector_details = {
                'type': vector['VectorType'],
                'description': self.get_vector_description(vector['VectorType'])
            }
            characterization['attack_vectors'].append(vector_details)

        # Get CloudWatch metrics for the attack
        metrics = self.get_attack_metrics(
            attack['StartTime'],
            attack.get('EndTime', datetime.now())
        )
        characterization['metrics'] = metrics

    return characterization
except Exception as e:
    return {'error': str(e)}

def classify_attack(self, attack):

```

```

"""Classify attack as Layer 3, 4, or 7"""

vectors = [v['VectorType'] for v in attack['AttackVectors']]

classification = {
    'layer_3': [],
    'layer_4': [],
    'layer_7': []
}

for vector in vectors:
    if 'UDP' in vector or 'ICMP' in vector or 'DNS' in vector:
        classification['layer_3'].append(vector)
    elif 'SYN' in vector or 'ACK' in vector:
        classification['layer_4'].append(vector)
    elif 'HTTP' in vector or 'HTTPS' in vector:
        classification['layer_7'].append(vector)

# Determine primary attack layer
if classification['layer_7']:
    return 'Layer 7 (Application) - HTTP/HTTPS Flood'
elif classification['layer_4']:
    return 'Layer 4 (Transport) - Protocol Attack'
else:
    return 'Layer 3 (Network) - Volumetric Attack'

def get_vector_description(self, vector_type):
    """Get human-readable description of attack vector"""

    descriptions = {
        'UDP_FLOOD': 'Large volume of UDP packets sent to target',
        'SYN_FLOOD': 'Large volume of SYN packets to exhaust connection states',
        'DNS_QUERY_FLOOD': 'Large volume of DNS queries (amplification attack)',
        'HTTP_FLOOD': 'Large volume of HTTP requests to exhaust application',
        'HTTPS_FLOOD': 'Large volume of HTTPS requests with SSL overhead',
        'ACK_FLOOD': 'Large volume of ACK packets',
        'ICMP_FLOOD': 'Large volume of ICMP packets',
        'NTP_REFLECTION': 'Abuse of NTP servers to amplify attack',
    }

```

```

        'SSDP_REFLECTION': 'Abuse of SSDP for amplification',
        'CHARGEN_REFLECTION': 'Abuse of Chargen service for amplification'
    }

    return descriptions.get(vector_type, f'Attack using {vector_type}')

def get_attack_metrics(self, start_time, end_time):
    """Get CloudWatch metrics during attack period"""

    metrics = {}

    metric_names = [
        'DDoS Attacked Bits',
        'DDoS Attacked Packets',
        'DDoS Attacked Requests'
    ]

    for metric_name in metric_names:
        try:
            response = self.cloudwatch.get_metric_statistics(
                Namespace='AWS/DDoSProtection',
                MetricName=metric_name,
                StartTime=start_time,
                EndTime=end_time,
                Period=60,
                Statistics=['Sum', 'Maximum', 'Average']
            )

            if response['Datapoints']:
                metrics[metric_name] = {
                    'peak': max([dp.get('Maximum', 0) for dp in response['Datapoints']]),
                    'average': sum([dp.get('Average', 0) for dp in response['Datapoints']])
                    'total': sum([dp.get('Sum', 0) for dp in response['Datapoints']])
                }
            except:
                pass

    return metrics

```

```

def calculate_severity(self, attack):
    """Calculate attack severity (low, medium, high, critical)"""

    severity_score = 0

    # Longer duration = higher severity
    duration = (attack.get('EndTime', datetime.now()) - attack['StartTime']).total_seconds()
    if duration > 3600:
        severity_score += 40 # >1 hour = high severity
    elif duration > 600:
        severity_score += 20 # >10 min = medium

    # Multiple attack vectors = higher severity
    vector_count = len(attack['AttackVectors'])
    if vector_count > 3:
        severity_score += 30
    elif vector_count > 1:
        severity_score += 15

    # Layer 7 attacks = higher severity
    if any('HTTP' in v['VectorType'] for v in attack['AttackVectors']):
        severity_score += 25

    # Determine level
    if severity_score >= 70:
        return 'CRITICAL'
    elif severity_score >= 50:
        return 'HIGH'
    elif severity_score >= 25:
        return 'MEDIUM'
    else:
        return 'LOW'

def analyze_impact(self):
    """Analyze business and technical impact"""

    impact_analysis = {

```

```
'availability_impact': {},
'performance_impact': {},
'cost_impact': {},
'user_impact': {}
}

# Get application availability during attack
# (This would require application-specific monitoring)
impact_analysis['availability_impact'] = {
    'uptime_percentage': '99.9%', # Assuming Shield mitigated effectively
    'downtime_minutes': 1,
    'services_affected': ['API', 'Web Portal'],
    'status': 'Mitigated successfully by Shield Advanced'
}

# Get performance metrics
impact_analysis['performance_impact'] = {
    'latency_increase': '45%', # 45ms normal → 65ms during attack
    'request_time_p99': '125ms',
    'error_rate': '0.1%',
    'capacity_utilization': '85%'
}

# Estimate cost impact
impact_analysis['cost_impact'] = {
    'data_transfer_cost': 500, # Waived by Shield Advanced
    'shield_advanced_protection': 'Active - Costs Covered',
    'estimated_cost_without_shield': 2000,
    'cost_savings': 2000
}

# Estimate user impact
impact_analysis['user_impact'] = {
    'affected_users': 5000,
    'support_tickets': 45,
    'affected_transactions': 200,
    'transaction_value': 50000, # $50k in transactions impacted
    'user_experience': 'Brief latency, no service loss'
```

```

    }

    return impact_analysis

def root_cause_analysis(self):
    """Analyze root causes and attacker intent"""

    rca = {
        'attack_origin': {},
        'attacker_sophistication': 'MEDIUM',
        'attacker_intent': [],
        'targeting_method': 'Likely: Public IP scanning',
        'attack_pattern': 'Multi-vector, coordinated attack'
    }

    # Analyze attack patterns
    rca['attack_origin'] = {
        'primary_sources': '50+ unique source IPs',
        'geographic_distribution': 'Worldwide (likely botnet)',
        'source_types': ['Data centers', 'Residential ISPs', 'Cloud providers'],
        'coordination': 'Synchronized attack pattern'
    }

    rca['attacker_intent'] = [
        'Ransom demand (likely)',
        'Competitor sabotage (possible)',
        'Testing/reconnaissance (low probability)',
        'Ideological motivation (possible)'
    ]

    return rca

def prevention_recommendations(self):
    """Generate prevention and improvement recommendations"""

    recommendations = {
        'immediate_actions': [
            'Review attack logs for attacker contact information',

```

```

        'Analyze attack patterns to update WAF rules',
        'Consider geo-blocking high-risk regions if applicable',
        'Increase rate limits to handle future attacks'
    ],
    'waf_improvements': [
        {
            'rule': 'HTTP flood rate limiting',
            'recommendation': 'Lower threshold from 5000 to 2000 requests/5min',
            'impact': 'May increase false positives, requires testing'
        },
        {
            'rule': 'Custom bot detection',
            'recommendation': 'Implement advanced bot control rules',
            'cost': '$10/month per 1M requests'
        }
    ],
    'shield_advanced_upgrades': [
        'Already enabled - Continue subscription',
        'Enable DRT access to emergency hotline',
        'Schedule quarterly DRT architectural review'
    ],
    'architecture_improvements': [
        'Increase edge location capacity',
        'Add geographic redundancy',
        'Implement blue-green deployment for failover',
        'Setup multi-region failover'
    ],
    'operational_improvements': [
        'Update incident response playbook',
        'Conduct DDoS drill/tabletop exercise',
        'Train operations team on Shield Advanced features',
        'Setup automated escalation procedures'
    ]
}

return recommendations

```


Usage

```
investigator = DDoSInvestigator(attack_id='12345678-1234-1234-1234-123456789012')
```

Conduct full investigation

```
investigation = investigator.investigate_ddos_attack()
```

Generate report

```
print("\n=== DDoS Attack Investigation Report ===")
print(json.dumps(investigation, indent=2, default=str))
```

Generate executive summary

```
print("\n=== Executive Summary ===")
summary = {
    'attack_type': investigation['phases']['characterization']['attack_type'],
    'duration': f"{investigation['phases']['assessment']['duration_seconds']/60:.1f} minutes",
    'severity': investigation['phases']['characterization']['estimated_severity'],
    'status': 'Mitigated',
    'cost_impact': investigation['phases']['impact']['cost_impact']['cost_savings'],
    'user_impact': 'Minimal (99.9% uptime maintained)'
}
print(json.dumps(summary, indent=2))
```

References

[1] AWS WAF User Guide - AWS Documentation. <https://docs.aws.amazon.com/waf/latest/developerguide/>

[2] AWS Managed Rules for AWS WAF - AWS Documentation. <https://docs.aws.amazon.com/waf/latest/developerguide/aws-managed-rule-groups.html>

[3] AWS Shield - Managed DDoS Protection - AWS Documentation. <https://docs.aws.amazon.com/shield/latest/userguide/>

[4] OWASP Top 10 mapped to AWS Managed Rules - AWS Community Builders. <https://dev.to/aws-builders/owasp-top-10-mapped-to-aws-managed-rules-5h78>

[5] AWS WAF and Shield Pricing Explained - AWS for Engineers. <https://awsforengineers.com/blog/aws-waf-and-shield-pricing-explained/>

[6] Mitigating DDoS with AWS Shield - CBT Nuggets. <https://www.cbtnuggets.com/blog/technology/networking/mitigating-ddos-with-aws-shield>

[7] AWS WAF Best Practices and Configuration Guide - AWS Security Services Best Practices. <https://aws.github.io/aws-security-services-best-practices/guides/waf/configuring-waf-rules/>

[docs/](#)

[8] DDoS Protection Techniques and Solutions 2024 - Radware Cyberpedia. <https://www.radware.com/cyberpedia/ddospedia/ddos-protection-techniques-types-and-7-solutions-to-know-in-2024/>

[9] AWS WAF Rules - Official Documentation. <https://docs.aws.amazon.com/waf/latest/developerguide/waf-rules.html>

[10] AWS Shield Pricing - Official Pricing Page. <https://aws.amazon.com/shield/pricing/>